

Modelos de Difusão para Geração de Imagens

Prof. Anderson Rocha

27 de outubro de 2025

Agenda

Parte 1:

- ▶ Intuição: ruído, difusão e denoising (1D primeiro)
- ▶ Processo de *forward noising*: $q(\mathbf{x}_t \mid \mathbf{x}_0)$
- ▶ Score matching e DSM (denoising score matching)
- ▶ DDPM: objetivo de treinamento & predição de ε
- ▶ Exemplo: mistura Gaussiana 1D $\rightarrow$ difusão

Aula 2:

- ▶ *Sampling*: $p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$, DDPM vs DDIM
- ▶ Condicionamento (classifier guidance e classifier-free guidance)
- ▶ Do 1D para 2D (imagens): arquitetura U-Net + atenção
- ▶ Boas práticas, custos, riscos e limitações
- ▶ Exercícios guiados + soluções passo a passo

Motivação: por que difusão?

- ▶ Geração estável e de alta qualidade via processo de **denoising** iterativo.
- ▶ Objetivo: aprender a **reverter** um processo que *adiciona ruído* aos dados.
- ▶ Vantagens: treino estável, objetivo simples (MSE na predição de ruído), bons gradientes.
- ▶ Conexão teórica com **score matching** e SDEs (equações diferenciais estocásticas).

Começando em 1D: variável escalar $x \in \mathbb{R}$

Cenário:

1. Considere uma distribuição simples de dados: $x_0 \sim 0.5\mathcal{N}(-\mu, \sigma^2) + 0.5\mathcal{N}(\mu, \sigma^2)$.
2. A cada passo t , **adicionamos ruído Gaussiano** controlado por β_t .
3. Após muitos passos, $x_T \approx \mathcal{N}(0, 1)$ (ruído puro).

Intuição: aprenderemos a **prever o ruído** para desfazer a difusão passo a passo.

Processo de difusão (forward noising)

- ▶ Defina $\alpha_t = 1 - \beta_t$ e $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$.
- ▶ **Fato chave:** $q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)\mathbf{I})$.
- ▶ Assim, podemos amostrar **em um único passo**:
$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 1).$$
- ▶ Isso simplifica o treinamento: geramos pares (x_t, ε) *sem* simular toda a cadeia.

Objetivo de treinamento do DDPM

- ▶ Modelo $\varepsilon_{\theta}(x_t, t)$ aprende a **predizer o ruído** adicionado.
- ▶ Perda MSE: $\mathcal{L} = \mathbb{E}_{t, x_0, \varepsilon} [\|\varepsilon - \varepsilon_{\theta}(x_t, t)\|^2]$.
- ▶ Equivalente a aprender o **score** $\nabla_{x_t} \log q(x_t)$ (via reescalonamentos).
- ▶ **Decisão de projeto:** predição de ε (em vez de x_0 ou do score) simplifica e estabiliza.

Posterior Gaussiano: $q(x_{t-1} \mid x_t, x_0)$

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\left(\underbrace{\tilde{\mu}_t(x_t, x_0)}_{\text{média}}, \underbrace{\tilde{\beta}_t \mathbf{I}}_{\text{variância}}\right),$$
$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t,$$
$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t.$$

Uso prático: no *sampling*, substituímos x_0 por estimativa via

$$\hat{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta(x_t, t)).$$

Escolha do agendamento de β_t (*noise schedule*)

- ▶ **Linear** (simples e eficaz): β_t cresce linearmente $\rightarrow$ bom ponto de partida.
- ▶ **Cosine** (melhor cobertura tardia de ruído): melhora qualidade de amostras.
- ▶ **Aprendido** (p.ex., variância aprendida): mais flexível, custo extra.
- ▶ **Decisão didática**: usar *linear* na Parte 1; discutir alternativas na Aula 2.

Exemplo (1D): difusão de mistura Gaussiana

Passos para derivar ao vivo:

1. Fixe $T = 1000$, β_t linear de 10^{-4} a 0,02.
2. Mostre $\bar{\alpha}_t$ e o *fechamento* para $\mathcal{N}(0, 1)$ quando $t \rightarrow T$.
3. Derive $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon$.
4. Escreva a perda MSE e destaque o *reweighting* opcional por λ_t .

Insight: tudo que precisamos para treinar é amostrar t , gerar x_t e pedir ao modelo para prever ε .

Atenção à normalização dos dados

- ▶ Normalizar x_0 para $[-1, 1]$ (imagens) ou média zero/variância unidade (1D)
→ **gradientes estáveis**.
- ▶ Data augmentation em 2D pode ser aplicado *no input* x_0 .

Resumo da Parte 1

- ▶ Treinar = aprender a **prever o ruído** em diferentes t .
- ▶ Encerramos com um exemplo 1D totalmente derivado no quadro.
- ▶ Prévia da Aula 2: **amostragem**, orientação por rótulos/texto e extensão para 2D com U-Net.

Amostragem (DDPM): revertendo a difusão

$$\begin{aligned}x_{t-1} &= \tilde{\mu}_{\theta}(x_t, t) + \tilde{\sigma}_t z, \quad z \sim \mathcal{N}(0, 1), \\ \tilde{\mu}_{\theta}(x_t, t) &= \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_{\theta}(x_t, t) \right), \\ \tilde{\sigma}_t^2 &= \tilde{\beta}_t \text{ (ou variância aprendida).}\end{aligned}$$

- ▶ Inicia em $x_T \sim \mathcal{N}(0, 1)$ e itera até x_0 .
- ▶ Menos passos $\Rightarrow$ mais rápido, porém risco de *drift*.

DDIM: amostragem determinística

- ▶ Remove ruído estocástico no *sampling* (SDE $\rightarrow$ ODE determinística).
- ▶ Permite **subamostrar** passos (p.ex., de 1000 para 50–100) mantendo qualidade.
- ▶ Útil para **edição** e controle de trajetórias latentes.

Condicionamento por rótulos ou texto

- ▶ **Classifier guidance:** usa um classificador $p(y \mid x_t, t)$ para inclinar a trajetória.
- ▶ **Classifier-free guidance (CFG):** treina o gerador com e sem condição; na amostragem interpola: $\varepsilon_{\text{cfg}} = \varepsilon_{\theta}(x_t, y) - w \varepsilon_{\theta}(x_t, \emptyset)$.
- ▶ Parâmetro w controla fidelidade vs diversidade.

Do 1D ao 2D: arquitetura para imagens

- ▶ **U-Net** 2D com **embeddings de tempo** t (p.ex., posicional sinusoidal) injetados em blocos.
- ▶ **Atenção** (self/cross) em resoluções médias/baixas para *global context*.
- ▶ Normalizações (GroupNorm), *residual blocks*, *skip connections*.
- ▶ Para texto: **cross-attention** com embeddings do texto (p.ex., Transformer/CLIP/T5).

Boas práticas e decisões de projeto

- ▶ *Noise schedule*: começar linear; experimentar cosine.
- ▶ **Pesos de perda** por t (p.ex., $\lambda_t \propto (1 - \bar{\alpha}_t)$) podem balancear dificuldades.
- ▶ **EMA** dos pesos do modelo melhora estabilidade em amostragem.
- ▶ **FP16** e *gradient clipping* reduzem instabilidades.

Custos e limitações

- ▶ Amostragem relativamente **lenta** (muitos passos) $\Rightarrow$ *use DDIM/schedulers rápidos*.
- ▶ Viés de dados e segurança: saídas refletem o conjunto de treino.
- ▶ Memória e computação: U-Nets grandes requerem GPUs com VRAM alta.

Parametrização Reversa com ε_θ

$$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon \right)$$
$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right)$$

Parametrização Reversa com ε_θ

$$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon \right)$$
$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right)$$

Ideia

Em vez de prever x_0 , prever ε simplifica e estabiliza o treino.

O que é uma U-Net? (Intuição)

- ▶ Arquitetura de rede neural originalmente usada em segmentação de imagens.
- ▶ Estrutura em forma de “U”: **encoder (down)**, **bottleneck** e **decoder (up)**.
- ▶ Conecta partes de alta e baixa resolução com **skip connections**.

O que é uma U-Net? (Intuição)

- ▶ Arquitetura de rede neural originalmente usada em segmentação de imagens.
- ▶ Estrutura em forma de “U”: **encoder (down)**, **bottleneck** e **decoder (up)**.
- ▶ Conecta partes de alta e baixa resolução com **skip connections**.

Por que funciona bem na difusão?

Combina **informação global** (baixo nível de resolução) com **detalhes locais** (alto nível de resolução).

UNet e Difusão — Papel no Processo

- ▶ A U-Net recebe como entrada:
 - ▶ x_t — imagem ruidosa no passo t ;
 - ▶ t — passo de tempo embutido (time embedding);
 - ▶ Condições (ex.: embeddings de texto).
- ▶ A rede **prediz o ruído** $\varepsilon_\theta(x_t, t)$.
- ▶ Isso permite reconstruir: $\hat{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta)$.

UNet e Difusão — Papel no Processo

- ▶ A U-Net recebe como entrada:
 - ▶ x_t — imagem ruidosa no passo t ;
 - ▶ t — passo de tempo embutido (time embedding);
 - ▶ Condições (ex.: embeddings de texto).
- ▶ A rede **prediz o ruído** $\varepsilon_\theta(x_t, t)$.
- ▶ Isso permite reconstruir: $\hat{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta)$.

Resumo

ε_θ é o **único termo aprendido**, todo o resto da fórmula vem da matemática da difusão.

UNet no Forward e no Reverse

► Forward (treinamento):

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon$$
$$\varepsilon_{\theta}(x_t, t) \approx \varepsilon$$

► Reverse (geração):

$$x_{t-1} = \mu_{\theta}(x_t, t) + \sigma_t z$$
$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_{\theta}(x_t, t) \right)$$

UNet no Forward e no Reverse

► Forward (treinamento):

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon$$
$$\varepsilon_{\theta}(x_t, t) \approx \varepsilon$$

► Reverse (geração):

$$x_{t-1} = \mu_{\theta}(x_t, t) + \sigma_t z$$
$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_{\theta}(x_t, t) \right)$$

Ponto-chave

A U-Net **nunca gera a imagem diretamente**: ela fornece a estimativa de ruído usada para limpar x_t passo a passo.

Arquitetura da U-Net (detalhes práticos)

- ▶ Downsampling: convoluções e blocos residuais.
- ▶ Bottleneck: self-attention → capta contexto global.
- ▶ Upsampling: reconstrói detalhes com skip connections.
- ▶ Time embeddings injetados nos blocos.
- ▶ Cross-attention → incorpora texto ou outras condições.

Processo Forward (resumo)

$$q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, \beta_t I), \quad \alpha_t = 1 - \beta_t$$

$$q(x_t|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

Posterior e Reverse (resumo)

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(\tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$$
$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right)$$
$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z, \quad z \sim \mathcal{N}(0, I)$$

O que é uma U-Net? (intuição)

- ▶ Encoder (down) $\rightarrow$ bottleneck $\rightarrow$ decoder (up).
- ▶ **Skip connections** ligam resoluções altas e baixas.
- ▶ Self-attn em resoluções médias; cross-attn p/ texto.
- ▶ Embeddings de tempo injetados nos blocos.

- ▶ Entrada: x_t (imagem ruidosa), t (time embedding), condição (texto/classe).
- ▶ Saída: $\varepsilon_\theta(x_t, t) \Rightarrow$ estimativa do ruído em x_t .
- ▶ Reconstrução:

$$\hat{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta(x_t, t)).$$

Quem gera a imagem? U-Net $\neq$ gerador direto

Papel da U-Net

Prediz $\varepsilon_{\theta}(x_t, t) \Rightarrow$ **guia** do denoising.

Quem “desenha” de fato

A **equação de reversão**:

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_{\theta}(x_t, t) \right), \quad x_{t-1} = \mu_{\theta}(x_t, t) + \sigma_t z.$$

- ▶ Começamos em $x_T \sim \mathcal{N}(0, I)$ e vamos limpando passo a passo.
- ▶ No final, obtemos x_0 (imagem gerada).

- ▶ **Nevoeiro:** a U-Net é o “GPS” que diz a direção para sair do ruído.
- ▶ **Quem anda:** a equação de denoising que atualiza $x_t \rightarrow x_{t-1}$.
- ▶ Resultado: a imagem surge gradualmente conforme o nevoeiro dissipa.

Processo de Denoising: chamadas repetidas da U-Net

1. Inicialize $x_T \sim \mathcal{N}(0, I)$.
2. Para $t = T, \dots, 1$:
 - ▶ $\varepsilon_\theta^{(t)} = \text{U-Net}(x_t, t, \text{cond})$
 - ▶ $\mu_\theta \leftarrow$ fórmula acima
 - ▶ $x_{t-1} = \mu_\theta + \sigma_t z$

Observações

- ▶ Uma **chamada por passo**.
- ▶ Scheduler define quantos passos efetivos.

Quantas chamadas? Custos e aceleração

- ▶ Clássico: $T \sim 1000$ (DDPM) $\Rightarrow$ 1000 chamadas.
- ▶ Moderno: $T \sim 20\text{--}50$ (DDIM/schedulers) $\Rightarrow$ muito mais rápido.

Comparação

Modelo	Chamadas	Como gera
GAN	1	$x = G(z)$ direto
VAE	1	$x = \text{Dec}(z)$ direto
Difusão	T	denoising iterativo via ε_θ

Treinamento vs. Amostragem

- ▶ **Treinamento:** amostra $t \sim U\{1, \dots, T\}$ e faz **uma** chamada $\Rightarrow$ supervisiona $\varepsilon_{\theta}(x_t, t) \approx \varepsilon$.
- ▶ **Amostragem:** executa $\mathbf{T}$ chamadas sequenciais (um nível de ruído por vez).

Consequência

Treinar é relativamente barato; **gerar** é o gargalo $\Rightarrow$ motivação para DDIM, distillation e bons schedulers.

CFG e condicionamento – recordar é viver ;-)

$$\varepsilon_{\text{guid}} = \varepsilon_{\text{uncond}} + s(\varepsilon_{\text{cond}} - \varepsilon_{\text{uncond}})$$

- ▶ Cross-attn injeta texto na U-Net.
- ▶ $s \uparrow \Rightarrow$ mais fidelidade ao prompt, menos diversidade.

Resumo 1

- ▶ **U-Net não desenha pixels:** estima ε_θ .
- ▶ **Quem gera** é a equação $x_{t-1} = \mu_\theta + \sigma_t Z$.
- ▶ A U-Net é chamada **a cada passo** do denoising.
- ▶ Menos passos $\Rightarrow$ maior velocidade (DDIM/schedulers).

Resumo 2

- ▶ Forward: ruído progressivo com forma fechada.
- ▶ Posterior exato: produto de Gaussianas.
- ▶ Reverse: prever ε , loss simples.
- ▶ Geração: DDPM e DDIM.
- ▶ **U-Net: coração do processo de predição.**
- ▶ Condicionamento: CFG.

Como o texto entra no modelo

1. **Prompt textual** (ex.: “gato astronauta no espaço”).
2. **Tokenização** → sequência de tokens.
3. **Text encoder** (CLIP/T5) → embeddings de texto $E \in \mathbb{R}^{L \times d}$.
4. **Cross-attention** na U-Net:
 - ▶ Queries: mapas de ativação da imagem.
 - ▶ Keys/values: embeddings do texto.

Ideia

O texto guia a atenção da U-Net para padrões coerentes com o prompt.

Treinamento sem classificador (CFG)

- ▶ Treinamos a mesma U-Net com e sem condição textual.
- ▶ Com probabilidade p_{uncond} : prompt vazio ou “negative prompt”.
- ▶ Com probabilidade $1 - p_{\text{uncond}}$: prompt real.

$$\mathcal{L} = \mathbb{E}[\|\varepsilon - \varepsilon_{\theta}(x_t, t, \text{cond})\|^2]$$

Resultado

A rede aprende duas funções implícitas: $\varepsilon_{\text{uncond}}$ e $\varepsilon_{\text{cond}}$.

Amostragem com CFG — Passo a Passo

1. Obter $\varepsilon_{\text{uncond}}$ da U-Net (prompt vazio).
2. Obter $\varepsilon_{\text{cond}}$ da U-Net (prompt real).
3. Combinar com guidance scale s :

$$\varepsilon_{\text{guid}} = \varepsilon_{\text{uncond}} + s(\varepsilon_{\text{cond}} - \varepsilon_{\text{uncond}}).$$

4. Usar $\varepsilon_{\text{guid}}$ na equação reversa:

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_{\text{guid}} \right).$$

5. Atualizar x_{t-1} .

Exemplo Numérico Simplificado

- ▶ $\alpha_t = 0.9, \beta_t = 0.1, \bar{\alpha}_t = 0.6.$
- ▶ $\varepsilon_{\text{uncond}} = 0.2, \varepsilon_{\text{cond}} = 0.8.$
- ▶ $s = 3.$

Exemplo Numérico Simplificado

- ▶ $\alpha_t = 0.9, \beta_t = 0.1, \bar{\alpha}_t = 0.6.$
- ▶ $\varepsilon_{\text{uncond}} = 0.2, \varepsilon_{\text{cond}} = 0.8.$
- ▶ $s = 3.$

$$\varepsilon_{\text{guid}} = 0.2 + 3(0.8 - 0.2) = 2.0$$

$$\mu_{\theta} = \frac{1}{\sqrt{0.9}} \left(x_t - \frac{0.1}{\sqrt{0.4}} \times 2.0 \right)$$

Intuição

O texto não aparece diretamente na fórmula de denoising: ele muda ε_{θ} , que por sua vez muda μ_{θ} e o rumo da geração.

Efeito da guidance scale s

- ▶ s controla quão fortemente o texto influencia a imagem.
- ▶ s pequeno $\rightarrow$ mais diversidade, menos fidelidade ao prompt.
- ▶ s grande $\rightarrow$ mais fidelidade, menos diversidade e mais risco de artefatos.

Regra prática

$s \in [3, 7]$ costuma dar bons resultados na prática.

Resumo — CFG e Condicionamento Textual

- ▶ Texto é convertido em embeddings via encoder e injeta informação na U-Net.
- ▶ Treinamos a U-Net com e sem texto $\rightarrow$ duas trajetórias ($\varepsilon_{\text{cond}}$ e $\varepsilon_{\text{uncond}}$).
- ▶ Durante a amostragem, interpolamos essas trajetórias via guidance scale.
- ▶ Isso orienta a equação de difusão, sem precisar de classificador externo.

Exercício 1 (1D): derivar $q(x_t \mid x_0)$

Enunciado: Assuma $x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{\beta_t} \varepsilon_t$ com $\varepsilon_t \sim \mathcal{N}(0, 1)$ e x_0 fixo. Mostre que

$$q(x_t \mid x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_t} x_0, 1 - \bar{\alpha}_t\right).$$

Dica: usar indução e propriedades de Gaussianas.

Solução 1 — passo a passo

Base: para $t = 1$, $x_1 = \sqrt{\alpha_1}x_0 + \sqrt{\beta_1}\varepsilon_1$. Logo $\mathbb{E}[x_1 \mid x_0] = \sqrt{\alpha_1}x_0$ e $\text{Var}[x_1 \mid x_0] = \beta_1$.

Hipótese indutiva: suponha $x_t \mid x_0 \sim \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, 1 - \bar{\alpha}_t)$.

Passo:

$$x_{t+1} = \sqrt{\alpha_{t+1}}x_t + \sqrt{\beta_{t+1}}\varepsilon_{t+1}.$$

$$\mathbb{E}[x_{t+1} \mid x_0] = \sqrt{\alpha_{t+1}}\mathbb{E}[x_t \mid x_0] = \sqrt{\alpha_{t+1}}\sqrt{\bar{\alpha}_t}x_0 = \sqrt{\bar{\alpha}_{t+1}}x_0.$$

$$\begin{aligned}\text{Var}[x_{t+1} \mid x_0] &= \alpha_{t+1}\text{Var}(x_t \mid x_0) + \beta_{t+1} \\ &= \alpha_{t+1}(1 - \bar{\alpha}_t) + \beta_{t+1} \\ &= 1 - \alpha_{t+1}\bar{\alpha}_t = 1 - \bar{\alpha}_{t+1}.\end{aligned}$$

Conclui-se a forma fechada desejada.

Exercício 2 (1D): MSE na predição de ε

Enunciado: Mostre que minimizar $\mathbb{E}\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$ é (proporcionalmente) equivalente a maximizar uma $-\text{KL}(q \parallel p_\theta)$ apropriada do processo reverso. **Dica:** use $\hat{x}_0$ e substitua em $q(x_{t-1} \mid x_t, x_0)$.

Solução 2 — esboço

- ▶ Escreva a ELBO do DDPM e veja que todos os termos são constantes exceto o termo de reconstrução que leva ao MSE em ε .
- ▶ Use $\hat{x}_0(x_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta(x_t, t))$ para obter $\tilde{\mu}_\theta$.
- ▶ A diferença entre médias Gaussianas implica em um termo quadrático que é proporcional ao MSE em ε .

Exercício 3 (1D): amostragem DDIM

Enunciado: Derive as atualizações determinísticas do DDIM a partir da forma fechada de x_t e impondo trajetória com variância zero no reverso. **Dica:** defina um conjunto de tempos coarsen $\tau_1 > \dots > \tau_K$ e relacione $x_{\tau_{k-1}}$ e x_{τ_k} via $\hat{x}_0$.

Solução 3 — esboço

- ▶ Use $x_t = \sqrt{\bar{\alpha}_t} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$.
- ▶ Para DDIM, imponha ε compartilhado ao longo dos passos e retire o termo estocástico.
- ▶ Chega-se a $x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \varepsilon$ (determinístico dado $\hat{x}_0$ e ε fixo).

Exercício 4 (2D): do 1D para imagens

Enunciado: Esboce uma U-Net mínima para imagens 32×32 com canais (64, 128, 256), embeddings de tempo sinusoidais e um bloco de atenção na resolução 16×16 . Explique onde injetar t .

Solução 4 — guia

- ▶ Encoder: $[\text{Conv}3 \times 3 + \text{GN} + \text{SiLU}] \times 2 \rightarrow \text{Downsample} \rightarrow \text{repete por resoluções.}$
- ▶ Injeção de tempo: $\text{MLP}(\text{emb}(t))$ somado aos vieses dos blocos residuais.
- ▶ Bloco de atenção (self-attn) após os blocos em 16×16 .
- ▶ Decoder: espelha encoder com upsample e *skip connections*.
- ▶ Cabeça final: $\text{Conv}3 \times 3$ para prever ε com 3 canais (RGB).

Quadro 1: mistura Gaussiana 1D

1. Desenhe as duas Gaussianas iniciais e evolução de $q(x_t)$.
2. Derive $\bar{\alpha}_t$ a partir de α_t linear.
3. Escreva o algoritmo de treino minimalista (pseudo-código):

Para minibatches de x_0 :

```
amostrar  $t \sim \text{Uniform}(\{1, \dots, T\})$ 
```

```
amostrar  $\text{eps} \sim N(0, 1)$ 
```

```
 $x_t = \sqrt{\text{alphabar}_t} * x_0 + \sqrt{1 - \text{alphabar}_t} * \text{eps}$ 
```

```
 $L = || \text{eps} - \text{eps}_{\text{theta}}(x_t, t) ||^2$ 
```

```
atualizar  $\text{theta}$  por gradiente
```

Quadro 2: amostragem DDPM mínima

```
Entrada: T, { $\beta_t$ ,  $\alpha_t$ ,  $\gamma_t$ },  $\epsilon(\cdot)$   
 $x_T \sim N(0, I)$   
Para  $t = T, \dots, 1$ :  
     $\mu_t = \mu(x_t, t)$   
     $\sigma_t = (1/\sqrt{\alpha_t}) * (x_t - ((1-\alpha_t)/\sqrt{1-\alpha_t}) * \epsilon(\cdot))$   
     $\beta_t^2 = \beta_t$   
     $x_{t-1} = \mu_t + \sigma_t * z$ ,  $z \sim N(0, I)$  (se  $t > 1$ ; se  $t=1$ ,  $z=0$ )  
Sada:  $x_0$ 
```

Quadro 3: CFG (classifier-free guidance)

Dado condio y e peso w :

$_cfg = _(x_t, t, y) - w * _(x_t, t,)$

Usar $_cfg$ no lugar de $_$ no passo de sampling.

$$\begin{aligned}\alpha_t &= 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s, \\ q(x_t \mid x_0) &= \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)), \\ \hat{x}_0 &= \frac{1}{\sqrt{\bar{\alpha}_t}} (x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta(x_t, t)), \\ q(x_{t-1} \mid x_t, x_0) &= \mathcal{N}(\tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t), \\ \tilde{\mu}_\theta(x_t, t) &= \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right).\end{aligned}$$

- ▶ **DDPM**: Ho, Jain, Abbeel (NeurIPS 2020).
- ▶ **Score-based**: Song, Ermon (NeurIPS 2019) e Song et al. (ICLR 2021).
- ▶ **DDIM**: Song et al. (ICLR 2021).
- ▶ **Guidance**: Dhariwal, Nichol (NeurIPS 2021) e Ho, Salimans et al. (classifier-free guidance, 2022).